Project Report: Multi-Threaded Lab Simulation System

1. Objective

The primary objective of this project is to develop a robust, multi-threaded simulation of a computer-based testing environment. The system aims to model real-world resource constraints, specifically limiting the number of concurrent participants to simulate a physical laboratory with a fixed number of workstations. Furthermore, the project demonstrates the practical application of synchronization primitives, such as mutexes and semaphores, to maintain data integrity across shared memory and persistent storage.

2. Introduction

The Multi-Threaded Lab Simulation System is a console-based application designed to manage the end-to-end lifecycle of an academic quiz. The software features two distinct operational modes: an Admin Portal for content creation and a Student Portal for simulation execution. By utilizing a multi-threaded architecture, the system can simultaneously track individual student progress, update a live monitoring dashboard, and log results to an external file without interrupting the simulation flow.

3. Background

In modern operating systems, concurrency management is essential for applications that perform multiple tasks simultaneously. This project addresses three critical challenges in concurrent programming:

Mutual Exclusion: Ensuring that only one thread can modify shared data structures, such as the student status board, at any given time to prevent race conditions.

Resource Throttling: Limiting the number of threads that can access a specific resource (simulated computers) to prevent system saturation.

Serialized I/O: Managing access to a single file so that multiple threads do not overwrite each other's data during logging operations.

4. Platform and Languages

Programming Language: The system is implemented in C++, utilizing standard libraries for input/output and file management.

Concurrency Library: The POSIX Threads (pthread) library is used for thread creation, management, and synchronization.

Synchronization Tools: The implementation utilizes pthread_mutex_t for mutual exclusion and sem_t for counting and binary semaphores.

Operating Environment: The application is designed for Unix-like environments, utilizing ANSI escape codes for real-time console manipulation and screen clearing.

5. Methodology

5.1 Data Architecture
The system utilizes two primary structures to manage data:
Question Structure: Stores the question text, four possible options, and the integer representing the correct answer.
Student Structure: Maintains metadata for each participant, including their unique ID, current score, progress through the quiz, and current operational status.

5.2 Synchronization Implementation
Three primary primitives are deployed to regulate the simulation:
Global Mutex (g_lock): This lock is acquired whenever a thread reads from or writes to the shared student board.
Counting Semaphore (g_computerSem): Initialized with a value of 3, this semaphore ensures that only three student threads can be in the "Answering" state concurrently.
Binary Semaphore (g_fileSem): This semaphore acts as a lock for the quiz_results.txt file, ensuring atomicity during the logging of student choices.

5.3 Execution Logic
Administrative Setup: The admin defines three questions, providing text and four options for each.
Thread Initialization: The system spawns a monitor thread and a user-defined number of student threads.
Simulation Loop: Student threads wait for a computer slot, simulate thinking time using usleep, select a random answer, and log the results.
Monitoring: The monitor thread provides a live dashboard at the top of the console, refreshing at 150ms intervals to show class averages and individual progress.

6. Results
6.1 Simulation Performance
During execution, the system successfully managed up to 15 concurrent student threads. The counting semaphore correctly queued students, ensuring that while the lab was "full" (3 students), others remained in a "Waiting" status. The class average was dynamically updated as students finished, providing a real-time metric of group performance.

6.2 Data Integrity
The persistence layer functioned as intended, with the quiz_results.txt file containing a clean, serialized log of every student's choice. Because of the binary semaphore, no lines in the text file were interleaved or corrupted, despite multiple threads attempting to write to the file within milliseconds of each other.

6.3 Metric Comparisons
The Concurrency Limit was strictly enforced at 3 students by g_computerSem. Score Accuracy remained consistent, with correct answers awarded 20 points per question. UI Stability was maintained by the monitor thread, providing a flicker-free display using ANSI positioning.

7. Conclusion

The Multi-Threaded Lab Simulation System successfully demonstrates the principles of concurrent process management within a controlled environment. By integrating mutexes and semaphores, the application avoids the common pitfalls of parallel programming, such as race conditions and resource starvation. The architecture proves that even in a restricted resource scenario, efficient thread scheduling and synchronization can ensure data consistency and provide a seamless user experience